

Project Interim Report
On
File Compression Simulator
(Using Core Java & OOP Concepts)

RU-100-01-00012
Object Oriented Programming With Java

SESSION: 2025-26



Submitted By
Shreyansh Patel
ERP no. 11395

Engineering in Computer Science with AI
In association with Google

Under the Guidance of
Mr. Ashish Shivhare

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI , CG

(April 2026)

PROJECT TITLE -

Smart Warehouse Grid Locator

Abstract -

The Smart Warehouse Grid Locator is a Java-based application designed to simulate and manage inventory within a physical warehouse space. Modern logistics require rapid item retrieval; this project addresses that need by representing the warehouse as a 2D grid structure. By utilizing Object-Oriented Programming (OOP), each item is treated as an object containing specific metadata like ID and Name . The system allows users to store items at precise coordinates and retrieve their exact location through an efficient search algorithm . This implementation serves as a foundational tool for automating spatial mapping in inventory management.

Introduction -

As warehouses grow in size, manual tracking of stock becomes prone to error. This project implements a digital twin of a warehouse floor using a 2D Array of objects.

Core Features:

- **Spatial Mapping:** Precise tracking using Row and Column coordinates.
- **Object Encapsulation:** Item details are protected within class structures.

Core Features:

- **Spatial Mapping:** Precise tracking using Row and Column coordinates.
- **Object Encapsulation:** Item details are protected within class structures.
- **Grid Visualization:** Ability to view the entire warehouse layout.

Objectives:

- To represent a physical warehouse as a digital **2D grid**.
- To store and manage **GridItem objects** within specific array indices .
- To implement a **search algorithm** using nested loops for coordinate retrieval.
- To reduce the time required to locate items in a large-scale environment.

Scope

This project is suitable for small-to-medium retail stockrooms or parts warehouses. While currently a console-based simulation, the architecture is designed to be **modular and scalable**, allowing for future integration with hardware like barcode scanners or database systems.

System Requirements

- **Hardware:** Computer with minimum 4GB RAM.
- **Software:** Java JDK (v8 or higher), IDE (VS Code, Eclipse, or IntelliJ).

Methodology

The system follows a structured execution flow:

1. **Initialization:** The Warehouse class initializes a 2D array of a defined size (e.g., 5×5) .

2. **Population:** GridItem objects are created using constructors and assigned to specific grid[row][column] positions .
3. **Search Request:** The user inputs an Item ID to find.
4. **Traversal:** The system uses **nested loops** (i and j) to check every cell in the 2D array.
5. **Output:** If the ID matches, the system returns the Row and Column index; otherwise, it reports "Item not found" .

Flowchart & Class Diagram

Class Structure:

GridItem Class	Warehouse Class
Attributes: itemID, itemName, quantity	Attributes: GridItem[][] grid
Methods: Constructor(), getItemID()	Methods: searchItem(String id), displayGrid()

Implementation

- **GridItem Class:** This class focuses on Encapsulation. It stores the data for a single inventory unit.
-
- **Warehouse Class:** This is the core logic engine. It manages the 2D array handling. The search function employs a Linear Search across two dimensions, comparing the target ID with the itemID attribute of the object stored at each grid coordinate.

Sample Input/Output

Test Case 1:

Successful Search

- **Input:** `warehouse.searchItem("I102");`
- **Output:** Searching for item ID: 1102... Item found at Row: 1, Column: 2
-

Test Case 2:

- **Item Not Found**
- **Input:** `warehouse.searchItem("I999");`
- **Output:** Item not found in warehouse
- 9.

Future Enhancements

- **Dynamic Input:** Allow users to add items via the console during runtime.
-
- **Inventory Removal:** Implement functionality to clear a grid cell when an item is shipped.
-
- **GUI Integration:** Transition from a text-based grid to a visual graphical interface.

Conclusion

The Smart Warehouse Grid Locator successfully demonstrates the power of OOP and 2D arrays in solving spatial problems. By organizing data into a grid, the system provides a clear, logical method for managing inventory that mimics real-world warehouse operations.

References

- H. Schildt, Java: The Complete Reference, 11th ed. New York: McGraw-Hill, 2018.
- [2] Oracle, "Java Documentation: 2D Arrays," Oracle. [Online]. Available: <https://docs.oracle.com>